

Privacy-First Personal Finance Agent: ReceiptIQ

Individual Project Report Member 7

Role: QA & Documentation Specialist

Name:	Baldev Singh Jadon
Roll No.:	24MC3012
Department:	Mathematics and Computing
Programme:	B.Tech MnC
Course:	Artificial Intelligence
Supervisor:	Dr. Shusham Biswas
Date:	April 19, 2026

Contents

1 Introduction	2
1.1 Role Summary	2
2 Quality Assurance & Testing	2
2.1 Testing Areas	2
2.2 Edge Cases	2
3 Documentation Strategy	3
4 System Resilience Implementation	3
5 Key Learnings	4
6 Conclusion	5

Introduction

Role Summary

This report documents the contribution of Member 7, who served as the QA & Documentation Specialist for the ReceiptIQ project. My responsibilities included ensuring the system remained stable, accurate, and easy to maintain. This involved designing testing flows for OCR reliability, validating module integration, handling edge cases gracefully, and producing comprehensive documentation for future maintainability.

Quality Assurance & Testing

Testing Areas

To ensure a robust privacy-first application, rigorous testing was required across the entire modular pipeline. The primary testing areas included:

- OCR reliability across different receipt types
- Classification flow
- Module integration
- Export functionality
- UI interaction flow

Edge Cases

Real-world receipt images are highly variable and often imperfect. Testing specifically targeted the following edge cases to ensure the system did not break under sub-optimal conditions:

- Low-quality images
- Partial text extraction
- Invalid formats
- OCR failures

The primary goal during edge case testing was to guarantee graceful failure handling rather than application crashes.

Documentation Strategy

To ensure the project's longevity and usability, I created comprehensive documentation. This structured knowledge base ensures the project is reliable and ready for extension. Key areas covered include:

- Architecture
- Usage instructions
- Feature explanations
- Future maintainability

System Resilience Implementation

Error Handling Strategy

Listing 1: Pseudo-code for Graceful Edge Case Handling

```
import logging

def process_receipt_image(image_path):
    try:
        # Attempt OCR extraction
        raw_text = extract_text_with_easyocr(image_path)

        if not raw_text or len(raw_text.strip()) == 0:
            raise ValueError("Partial or no text extracted.")

        return structure_data(raw_text)

    except ValueError as ve:
        logging.warning(f"Validation Error: {ve}")
        return {"status": "error", "message": "Could not read receipt."}

    except Exception as e:
        logging.error(f"Unexpected OCR Failure: {e}")
        return {"status": "error", "message": "System encountered an issue. Graceful exit."}
```

Python

Table 1: QA Scenarios and Resolution Strategies

Scenario	Resolution Strategy	Justification
Blurry/Wrinkled Image	Implement robust preprocessing triggers	Maximizes EasyOCR's extraction capabilities before failure
Complete OCR Failure	Trigger graceful error message in Gradio UI	Prevents backend crashes; guides user to take a better photo
Invalid File Format	Pre-upload validation checks	Conserves local processing power by rejecting unsupported files early

Key Learnings

1. Anticipating real-world messiness (blurry images, poor lighting) is just as critical as the core machine learning logic in OCR-based applications.
2. Graceful failure handling builds trust; an application that tells you why it can't read a receipt is far superior to one that simply crashes.
3. Documentation is not an afterthought; it is the bridge that makes a complex pipeline of OCR, ML, and Analytics accessible and maintainable.
4. End-to-end testing is vital in a modular architecture to ensure data flows correctly from frontend upload to backend local storage.

Conclusion

I designed and executed the QA and Documentation framework for the ReceiptIQ pipeline. By identifying edge cases, validating integration across all developer modules, and ensuring graceful failure handling, the application achieves its goal of being a stable, user-friendly, privacy-first personal finance agent. Furthermore, the extensive documentation ensures the system can be easily understood and extended by future developers.

Individual Contribution Statement: This report describes my independent contribution (QA & Documentation) to the group project.